

AFRILANG Stdlib

std/c/mlnaivebayes

Bibliothèque AFRILANG `std/c/mlnaivebayes` (niveau complex, catégorie complex). Elle expose 6 fonction(s)
: classPrior,

```
`import "std/c/mlnaivebayes" . `use mlnaivebayes`
```

- `classPrior(labels list number, cls number) ? number`
- `featureMean(features list number, labels list number, cls number, feat number) ? number`
- `featureVar(features list number, labels list number, cls number) ? number`
- `gaussianLikelihood(x number, mu number, sigma number) ? number`
- `predictClass(features list number, trainX list number, trainY list number, numClasses number) ? number`
- `logPosterior(features list number, trainX list number, trainY list number, cls number) ? number`

Category: complex | Tier: complex | Functions: 6

FRANCAIS

1. Vue d'ensemble

Bibliothèque AFRILANG `std/c/mlnaivebayes` (niveau complex, catégorie complex). Elle expose 6 fonction(s) : classPrior,

```
`import "std/c/mlnaivebayes" . `use mlnaivebayes`  
- `classPrior(labels list number, cls number) ? number`  
- `featureMean(features list number, labels list number, cls number, feat number) ? number`  
- `featureVar(features list number, labels list number, cls number) ? number`  
- `gaussianLikelihood(x number, mu number, sigma number) ? number`  
- `predictClass(features list number, trainX list number, trainY list number, numClasses number) ?  
number`  
- `logPosterior(features list number, trainX list number, trainY list number, cls number) ?  
number`
```

2. Installation & import

Ajoutez le module à votre programme :

```
import "std/c/mlnaivebayes"  
use mlnaivebayes
```

Niveau : complex · Catégorie : Modules complex (c/)
Domaines avancés ? graphes, NLP, réseaux complexes.

3. Référence API

```
? classPrior(labels list number, cls number) ? number  
? featureMean(features list number, labels list number, cls number, feat number) ? number  
? featureVar(features list number, labels list number, cls number) ? number  
? gaussianLikelihood(x number, mu number, sigma number) ? number  
? predictClass(features list number, trainX list number, trainY list number, numClasses number) ?  
number  
? logPosterior(features list number, trainX list number, trainY list number, cls number) ? number
```

4. Exemples d'utilisation

```
import "std/c/mlnaivebayes"  
use mlnaivebayes  
  
create result = classPrior(/* args */)   
say result  
  
create result = featureMean(/* args */)   
say result  
  
create result = featureVar(/* args */)   
say result  
  
create result = gaussianLikelihood(/* args */)   
say result  
  
create result = predictClass(/* args */)   
say result  
  
create result = logPosterior(/* args */)   
say result
```

5. Bonnes pratiques

- ? Préférez `import "std/c/mlnaivebayes"` en tête de fichier.
- ? Lancez `afrilang check` avant de livrer.
- ? Combinez avec les modules stdlib de la même catégorie.
- ? Voir /stdlib/ pour le source live et les démos playground.
- ? Signalez les problèmes sur GitHub si une export manque.

6. Annexe ? source

```
module mlNaiveBayes

function classPrior(labels list number, cls number) returns number
end

function featureMean(features list number, labels list number, cls number, feat number) returns
number
end

function featureVar(features list number, labels list number, cls number) returns number
end

function gaussianLikelihood(x number, mu number, sigma number) returns number
end

function predictClass(features list number, trainX list number, trainY list number, numClasses
number) returns number
end

function logPosterior(features list number, trainX list number, trainY list number, cls number)
returns number
end
end
```

ENGLISH

1. Overview

AFRILANG library `std/c/mlnaivebayes` (tier complex, category complex). Exports 6 function(s):
classPrior, featureMean,

```
`import "std/c/mlnaivebayes" . `use mlnaivebayes`  
- `classPrior(labels list number, cls number) ? number`  
- `featureMean(features list number, labels list number, cls number, feat number) ? number`  
- `featureVar(features list number, labels list number, cls number) ? number`  
- `gaussianLikelihood(x number, mu number, sigma number) ? number`  
- `predictClass(features list number, trainX list number, trainY list number, numClasses number) ?  
number`  
- `logPosterior(features list number, trainX list number, trainY list number, cls number) ?  
number`
```

2. Installation & import

Add the module to your program:

```
import "std/c/mlnaivebayes"  
use mlnaivebayes
```

Tier: complex · Category: Complex modules (c/)
Advanced domains ? graphs, NLP, complex networks.

3. API reference

```
? classPrior(labels list number, cls number) ? number  
? featureMean(features list number, labels list number, cls number, feat number) ? number  
? featureVar(features list number, labels list number, cls number) ? number  
? gaussianLikelihood(x number, mu number, sigma number) ? number  
? predictClass(features list number, trainX list number, trainY list number, numClasses number) ?  
number  
? logPosterior(features list number, trainX list number, trainY list number, cls number) ? number
```

4. Usage examples

```
import "std/c/mlnaivebayes"  
use mlnaivebayes  
  
create result = classPrior(/* args */)   
say result  
  
create result = featureMean(/* args */)   
say result  
  
create result = featureVar(/* args */)   
say result  
  
create result = gaussianLikelihood(/* args */)   
say result  
  
create result = predictClass(/* args */)   
say result  
  
create result = logPosterior(/* args */)   
say result
```

5. Best practices

```
? Prefer `import "std/c/mlnaivebayes"` at the top of your file.  
? Run `afrilang check` before shipping.  
? Combine with related stdlib modules from the same category.  
? See /stdlib/ for the live source and playground demos.  
? Report issues on GitHub if an export is missing or undocumented.
```

6. Appendix ? source

```
module mlNaiveBayes

function classPrior(labels list number, cls number) returns number
end

function featureMean(features list number, labels list number, cls number, feat number) returns
number
end

function featureVar(features list number, labels list number, cls number) returns number
end

function gaussianLikelihood(x number, mu number, sigma number) returns number
end

function predictClass(features list number, trainX list number, trainY list number, numClasses
number) returns number
end

function logPosterior(features list number, trainX list number, trainY list number, cls number)
returns number
end
end
```